This part involved building responsive and clean web pages using **HTML, CSS, and Jinja2 templating**.
Each template is dynamically rendered through **FastAPI**, ensuring that users can easily submit inputs, view generated comics panel-by-panel, and confirm comic export actions.
This milestone guarantees an engaging and seamless comic creation experience for all users.

Activity 4.1 Designing and Developing the User Interface

Set Up the Base HTML Structure

Developed **index.html** as the main entry point for user input.
Structured the form to capture:

- Story Prompt
- Main Character Name
- Setting
- Story Tone
- Art Style

Each field is properly labeled and grouped using semantic HTML elements.
The design uses a **scenic background image**, simple typography, and organized form layout to encourage creativity and focus on storytelling.
Navigation is minimal to maintain focus, with a direct flow from input to comic generation.

Design a Responsive Layout Using CSS

Embedded CSS styles were applied directly within each HTML file.
Layouts use **fixed width with centered content** to ensure readability across different screen sizes.
Consistent button styling with **hover effects** was implemented.
Input fields and result panels are designed with **padding, box shadows, and rounded corners** for a clean, modern comic-generation experience.
The light color scheme enhances visibility and promotes an inviting visual aesthetic.

Create Separate Pages for Each Core Functionality

Three **Jinja2-powered HTML templates** were created under the /templates folder:

- **index.html**:
  A user-friendly form where users input the story prompt, character name, setting, tone, and preferred art style.
  Submits data to the /generate FastAPI route via POST.

- **comic_preview.html**:
  Displays the AI-generated comic panel-by-panel.
  Each panel includes:
  - Title
  - Scene illustration (image)
  - Scene description (in italics)
  - Captions and narration text This allows users to review the full comic storyline sequentially.
- **export_success.html**:
  Displays a success message after the user downloads their comic as a PDF.
  Encourages users to return and create a new comic.

Templates are modular, cleanly separated by functionality, and UI transitions smoothly between comic creation and comic review.

Activity 4.2 Creating Dynamic Templates with FastAPI's Jinja2

Integrate Jinja2 Templating for Dynamic Content Rendering

- Used FastAPI's Jinja2Templates for template rendering:

```
from fastapi.templating import Jinja2Templates
templates = Jinja2Templates(directory="templates")
```

Each FastAPI route uses **TemplateResponse** to dynamically render HTML templates based on the user's interaction:

- / route renders index.html
- /generate route renders comic_preview.html
- /export-success route renders export_success.html

This dynamic binding ensures that the comic content (layout, images, text) updates automatically after user input.

Bind Backend Data to HTML Templates

➤ **index.html**

- Form elements use name="..." attributes that directly map to backend Form(...) parameters in FastAPI.
- Submits form data via POST to /generate.

➤ **comic_preview.html**

- Dynamically displays:
  - Panel number, title
  - Image generated for the panel
  - Scene description (in italics)
  - Captions and narration text
- Loops through the layout list using Jinja2 {% for panel in layout %} syntax.

➤ **export_success.html**

- Displays a confirmation message using Jinja2 variables.
- Receives the exported PDF path as context and optionally provides a button for returning to the homepage.

The frontend of ComicCraft is fully integrated with FastAPI, dynamically renders AI-generated comics, and provides a smooth user flow from prompt input to comic export.

Milestone 5. Deployment